OCP GLOBAL SUMMIT 2026 • OPEN PLATFORM FIRMWARE

Intel FSP 的下一代演进

Michael Kinney · Chad Tsai，Intel

时长 13 分 50 秒

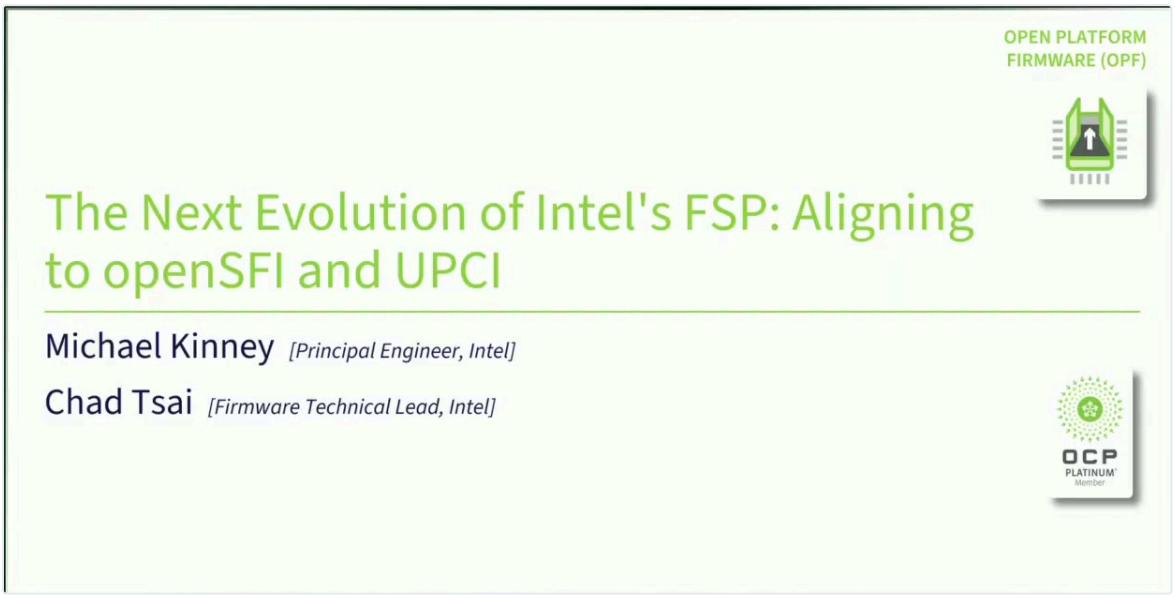
来源 youtube.com/watch?v=-wfc8OlYciU

页数 9 页幻灯片

Intel FSP 的下一代演进：向 openSFI 与 UPCI 对齐

> OCP Global Summit 2026 · Open Platform Firmware (OPF) 分会场 讲者：Michael Kinney（Intel 首席工程师）、Chad Tsai（Intel 固件技术负责人） 全长 13 分 50 秒 | 共 9 页幻灯片

01 开场：向 openSFI 与 UPCI 对齐



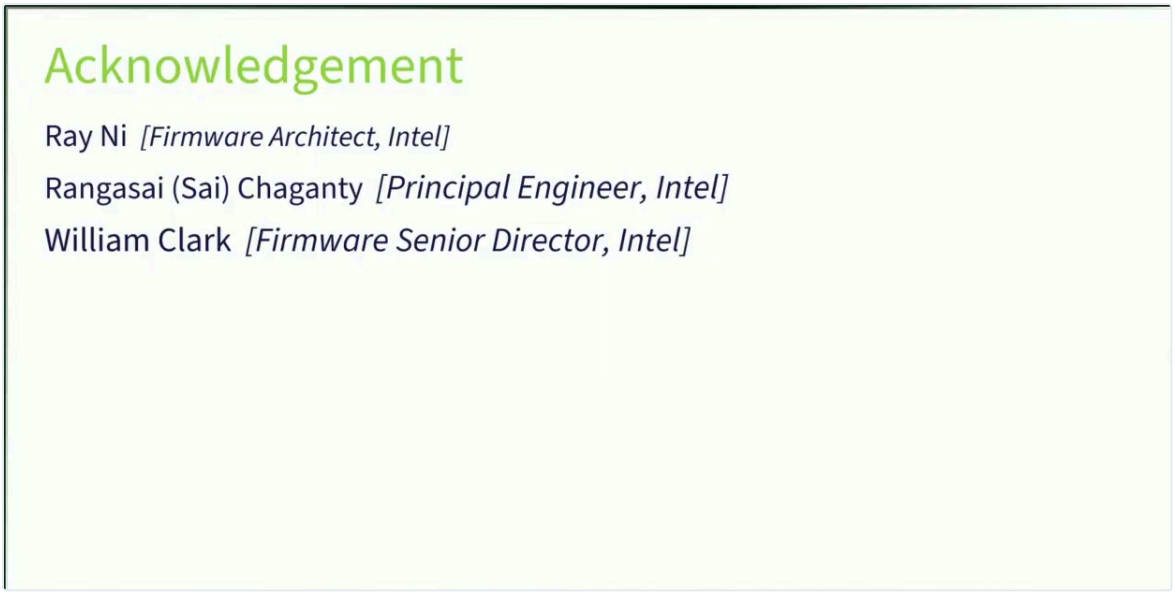
开场标题页：The Next Evolution of Intel's FSP: Aligning to openSFI and UPCI，讲者为 Michael Kinney 与 Chad Tsai（均为 Intel）

大家好。好，好。大家好，欢迎。呃，我叫 Chad，和我一起演讲的是 Mike。我们来自 Intel。好。呃，我们非常高兴来讨论 Intel FSP 的下一代架构演进。呃，具体来说，就是我们如何演进以对齐 OCP 这里正在推动的开放行业标准——OPF 的 openSFI，以及 UPCI。

> "Hello. Okay. Okay. Hello everyone, and welcome. Uh my name is Chad, and uh presenting with me is Mike. And we are from Intel. Okay. And um we are very excited to uh discuss our next uh architecture evolution of Intel FSP. Uh and that's specifically um how we are evolving to align with the open industry standard being driven here in OCP OPEF Open SFF and the UPCI."

> **小结** 这场演讲不是发布新规范，而是说明 Intel FSP 的实现如何向 OCP 的 openSFI 与 UPCI 靠拢——重点在"实现层对齐"，而非"规范变更"。

02　致谢



Acknowledgement 页：列出 Ray Ni（固件架构师）、Rangasai (Sai) Chaganty（首席工程师）、William Clark（固件高级总监）三位 Intel 贡献者

好。在进入技术细节之前，我们想先感谢为这项工作做出贡献的 Intel 各位关键负责人。呃，Ray Ni、Sai，以及 Bill Clark，当然，还有 Allison。是的。

> "Okay. So, before we jump into the technical details, we want to give credit to the key leaders in Intel who contribute to this work. Um Rainey, Sai, and the Bill Clark, of course, uh Allison. Yeah."

> **小结**　致谢页列出 Ray Ni、Rangasai (Sai) Chaganty、William Clark 三位核心贡献者——说明 FSP 的这次改造是 Intel 固件架构层面的团队工程，而非单一产品线改动。

---

03 议程



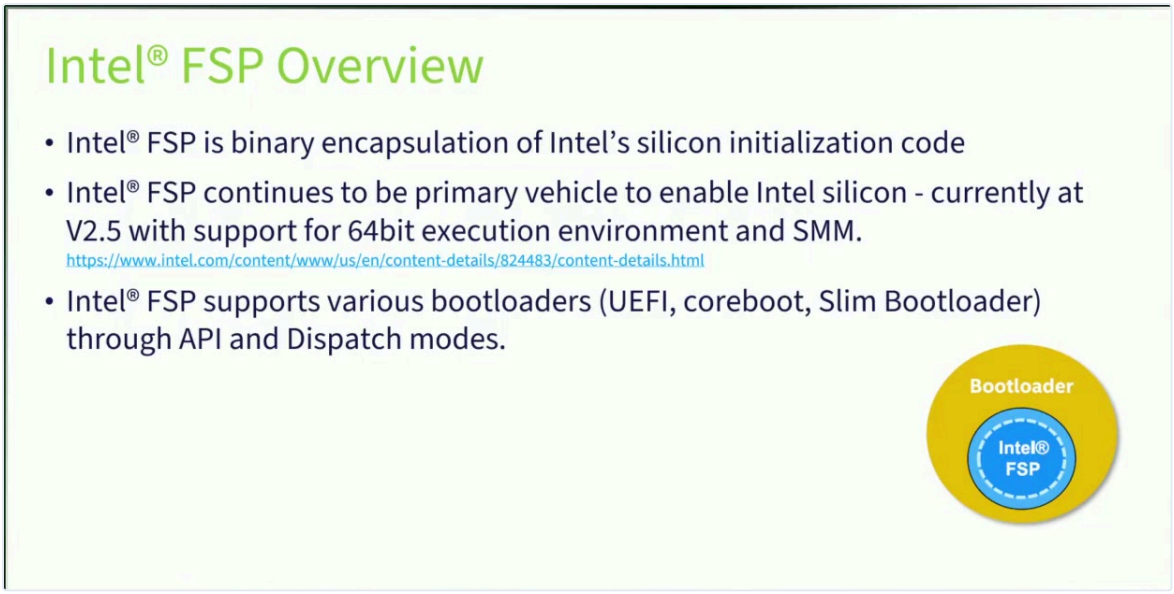
Agenda 页：Intel FSP Overview、Data Center Intel FSP Enhancement、openSFI and UPCI Alignment、Call To Action 四项

那么，这是今天的路线图。呃，首先我们从 Intel FSP 今天提供什么、以及它在各类引导加载程序（bootloader）中的基础性作用开始。然后我们会审视数据中心版 Intel FSP 的整体增强，并带大家过一遍我们下一代 Xeon 平台的高层架构变化。接着 Mike 会深入讲与 openSFI 的架构对齐，尤其是无状态（stateless）执行模型。最后我们以行动号召收尾，讲生态如何参与这些改进。好，我们先回顾一下……

> "So, here's the road map for uh today. Um first, we will begin with uh what Intel FSP provides today, and um its foundational role uh role uh across brothers. And then we will examine the overall data center Intel FSP uh enhancement, and we'll go you through uh the high-level architectural change uh for our next generation Xeon platforms. And then Mike will deep dive into the um architectural alignment with uh the Open SFF, especially the status execution model. And finally, we will close with a call to action on how the ecosystem can engage these improvements. Okay. let's start with a refresh on"

小结　四段式结构：现状（FSP 是什么）→ 数据中心增强（四大支柱 + 高层架构）→ 与 openSFI 的无状态模型对齐 → 生态行动号召。分工上 Chad 讲前两段，Mike 讲第三段。

04　Intel FSP 概览：二进制封装与两种引导模式



Intel FSP Overview：FSP 是 Intel 芯片初始化代码的二进制封装，当前为 V2.5（支持 64 位执行环境与 SMM），并通过 API 模式与 Dispatch 模式支持 UEFI、coreboot、Slim Bootloader 等引导程序

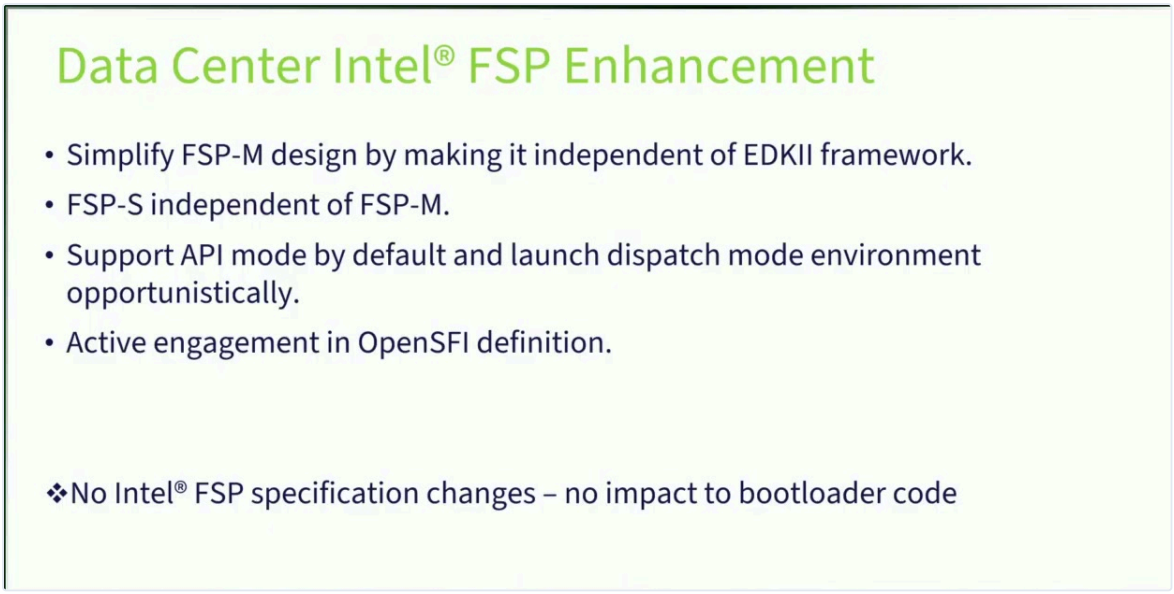
Intel FSP。Intel FSP 是 Intel 芯片初始化代码的二进制封装。它把我们的芯片编程 IP 打包成模块化、可直接消费的二进制。今天，Intel FSP 规范处于 2.5 版，该版本引入了对原生 64 位执行环境和 SMM 的正式支持。

那么，让 Intel FSP 格外有力的是它在各类引导加载程序之间的灵活性。所以今天，无论客户用 UEFI、coreboot 还是 Slim Bootloader 引导，FSP 都能通过两种不同的引导模式来支持：API 模式和 dispatch 模式。在 API 模式下，引导加载程序调用离散的初始化阶段，例如临时内存初始化（TempRamInit）、FSP 内存初始化（FspMemoryInit）或 FSP 芯片初始化（FspSiliconInit），通过这些 API 来调用。而在 dispatch 模式下，FSP 作为 PEIM 直接集成到 PI 框架里。这就是我们 FSP 的基础。

> "Intel FSP. Intel FSP is the binary encapsulation of Intel's compressed silicon in the initialization logic. It packages our silicon programming IP into a modular consumable binary. Today, the Intel FSP specification stands at version 2.5 which introduced introduced formal support on or for the native 64-bit execution environment and the SMM. So, what makes Intel FSP uniquely powerful is its flexibility across boot loaders. So, today whether a customer boots with a UEFI coreboot or the Slim Bootloader FSP supports through the two different boot modes, API mode and dispatch mode. So, for API mode the boot loader calls into discrete initialization phases. For example, the temporary init FSP memory init or the FSP silicon init. Those interface those APIs through those APIs. And then for dispatch mode FSP integrates directly as PI M into the PI foundation. This is a the basic our FSP foundation."

小结　FSP 的定位是"芯片初始化 IP 的二进制封装"，2.5 版补齐 64 位与 SMM。它同时支持 API 模式（引导程序调用离散阶段 API）与 dispatch 模式（作为 PEIM 嵌入 PI 框架）——这两种模式并存，正是后文"收敛执行流"要解决的复杂度来源。

05　数据中心版 FSP 增强的四大支柱



Data Center Intel FSP Enhancement：简化 FSP-M（脱离 EDK II PEI 框架）、FSP-S 独立于 FSP-M、默认 API 模式并按需启动 dispatch 模式、积极参与 OpenSFI 定义；并强调 FSP 规范不变、引导程序代码无需改写

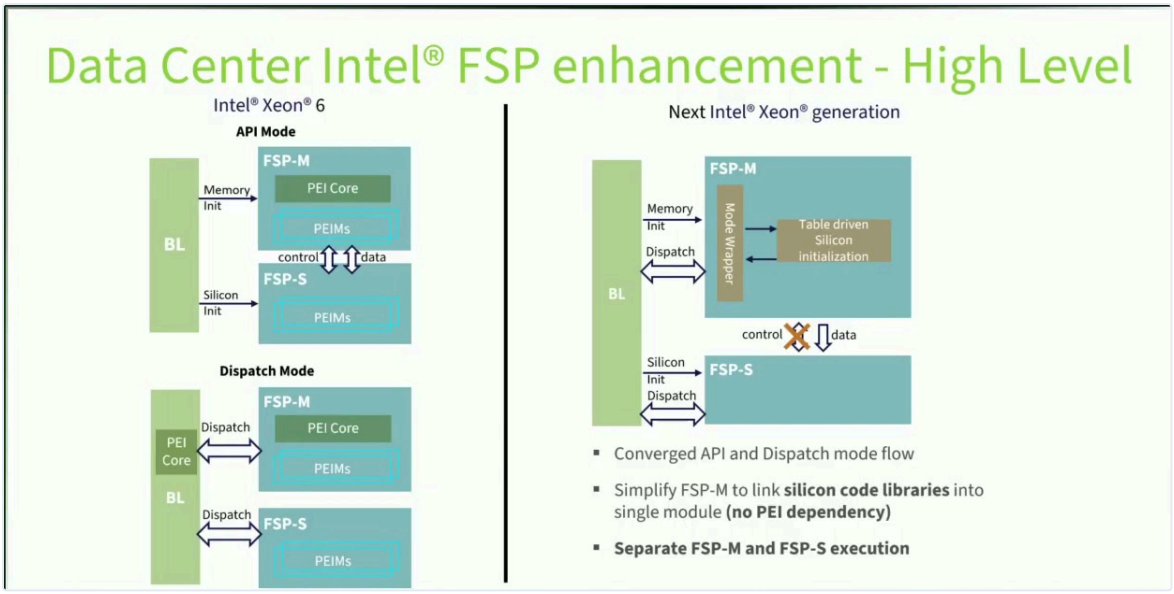
好，那么进入我们的主题，也就是增强。这里是我们 Intel FSP 增强的四大支柱。可以看到，第一个是我们简化 FSP-M：让 FSP-M 独立于 EDK II 的 PEI 框架。第二个是 FSP-S 解耦：FSP-S 独立于 FSP-M 运行，同时我们也消除了跨阶段的依赖。第三个是默认 API 模式：我们把 API 模式标准化为首要的原生执行流；同时我们仍然保留在需要时按需启动 dispatch 模式的能力。同时，我们也在积极参与 openSFI 的定义。

呃，但最重要的是：我刚才提到的所有 FSP 增强，都没有导致 Intel FSP 规范的变更；你们的引导加载程序代码也不需要破坏性重写，就能利用这些改进。

> "Okay, then move to the our um main topic that the enhancement. So, here's our the here are the four primary pillars of our Intel FSP enhancement. You can add the first one is the we simplify FSP M. We make the FSP M independent of EDK II PE mode. And second one is the we do the FSPS decoupling. Which is the FST runs independent of the independently of FSPM. We eliminating also eliminating the across phase dependencies. And third one is the AP mode by default. We standardize on API mode as our primary native uh, execution flow. Uh, at the same time we still um, while we still retaining uh, remaining the retaining the um, the capability uh, to launch the dispatch mode opportunity distically when needed. So, and also at the same time we're also um, actively engaging with the OSPi definition. So, um, but the most importantly uh, all the FSP enhancement I just mentioned didn't result in changes to our inter FSP specification. And also your boot loader code required no breaking rewrites to take advantage of these improvements."

> **小结**　四大支柱：①FSP-M 脱离 EDK II PEI 框架；②FSP-S 与 FSP-M 解耦、消除跨阶段依赖；③API 模式成为默认、dispatch 模式按需启用；④参与 openSFI 定义。关键在于"规范不变、引导程序无需破坏性改写"——改进落在实现层，兼容性得以保持。

06　架构对比：从 Xeon 6 到下一代 Xeon



Data Center Intel FSP enhancement - High Level：左侧为 Intel Xeon 6 的 API 模式与 Dispatch 模式（FSP-M/FSP-S 均内嵌 PEI Core 与 PEIMs），右侧为下一代 Xeon 的收敛流程（模块包装器 + 表驱动的芯片初始化，FSP-M 无 PEI 依赖，FSP-M/FSP-S 分离执行）

好。那我们来看这个架构转变是什么样子。好。呃，从左边的 Intel Xeon 6 架构开始。呃，先讲 API 模式。在 API 模式下，当引导加载程序调用 FSP-M 时，如你所见，FSP 会用内嵌的 PEI Core 来分发 PEIM。当过渡到 FSP-S 时，你同样依赖 PI 的基础设施与服务。这是我们的原有模型。呃，现在看 dispatch 模式：引导加载程序外层的 PI 代码会直接调用 FSP-M 和 FSP-S。呃，所以如你所见，这两种执行流高度取决于你选的是 API 模式还是 dispatch 模式。

现在看右边下一代 Xeon 平台的架构。呃，我们有三大更新。第一个是收敛的执行流（converged execution flow）。我们怎么做呢？我们把执行流统一了，呃，不再是两条彼此割裂的代码流——API 模式和 dispatch 模式共享同一套芯片初始化逻辑。这是第一个。第二个是去掉 PI 依赖的 FSP-M：呃，我们把 FSP-M 中旧的 PI 代码依赖移除，呃，把所有芯片核心库（silicon core libraries）链接进单个模块。好。那么，因为它是单个模块，它是代码表（code table）驱动的格式。呃，这样会非常便于阅读、也便于调试。第三个是我们把 FSP-M 与 FSP-S 的执行分离。好。呃，我们让 FSP-M 只跑在 NEM 上（也就是把缓存当作 RAM），而 FSP-S 只跑在物理内存上。同时我们把 PI 代码从 FSP-M 移到 FSP-S。这样一来，我们就不需要复杂的代码去修正从 NEM 到内存的指针；而且，FSP-S 到 FSP-M 之间不再有函数回调。唯一的一件事就是 FSP-M 到 FSP-S 的输出数据。这些就是我们在下一代 Xeon 平台上改动的东西。那么，我把时间交给 Mike，来讲与 openSFI 的架构对齐。Mike 是我们 Intel 的 openSFI 代表。欢迎 Mike。

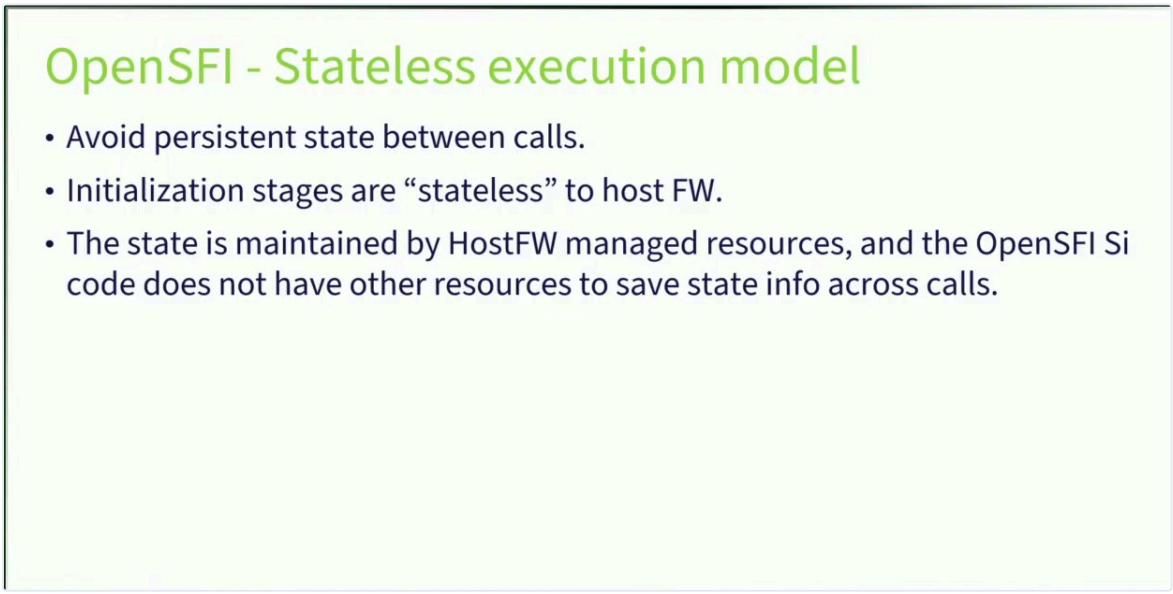
(Mike) 好，谢谢你 Chad。是的。谢谢你，Chad。

> "Okay. So, let's understand how this architecture shift looks. Okay. So, uh, begin with the uh, in the Xeon our Xeon Intel Xeon 6 uh, architecture shown on the left. Uh, let's talk about AP mode first. In the API mode when the boot loader calls uh, FSPM as you can see, FSP uh, will use an embedded PI core uh, to dispatch PIMs. When transition to the FSPS you again reacts on the PI infrastructure and services. This is our um original model. And the um now look at the uh dispatch mode. The bootloader um uh the bootloader's outer PI code will directly invoke the FSPM and FSPS directly. Uh So, as you can see, these two different execution flow highly depending on whether you choose AP mode or dispatch mode. And now look at the architecture on the um right-hand side for our next generation Xeon platform. Um

> we have three major updates. So, the first one is that um converged execution flow. How we're doing it is that um we have unified uh the execution flow uh and the um we um instead of two distinct disjointed um code flow the AP mode and the dispatch mode share the same silicon initialization logic. This is the first one and second one is that um it's the PI uh FSPM. uh so we make the uh FSPM um we do remove the older dependency PI PI code dependency from FSPM. And so uh and the we have all the silicon libraries silicon core libraries uh linked into a single module. Okay? So, with this uh because this is a single module it is a code code table based format. So, uh, so it will match the this very makes users to be easy to read and simple to debug. And the third one is the um we are separating FSPM and FSCS execution. Okay. Uh, we make the FSPM runs on NEM which is a cache as a RAM only and the FSCS runs on physical memory only. And we also move the PI code from FSPM to FSPS. With this, um we don't need a complicated code to fix up the pointer from name to memory. And also, there will be no function function callback from FSCS to FSPM. The only one thing is that the output data from FSPM to FSCS. It's those are what do we change it in our next generation Xeon platform. So, and I will hand it over to Mike to talk about the the architecture alignment with the Open SFI. Mike is our Open SFI Intel representative. So, welcome Mike. All right. Thank you, Chad. Yeah. Thank you, Chad."

> **小结**　三项改动：①收敛执行流——API 与 dispatch 共享同一套芯片初始化逻辑；②FSP-M 去 PI 依赖，芯片核心库链接为单一模块、表驱动、便于阅读调试；③FSP-M（仅 NEM/缓存当 RAM）与 FSP-S（仅物理内存）分离执行，PI 代码移入 FSP-S，从而取消 NEM→内存的指针修正与 FSP-S→FSP-M 的回调，只剩单向输出数据。

07　OpenSFI 无状态执行模型



OpenSFI - Stateless execution model：调用之间不留持久状态；初始化阶段对主机固件是"无状态"的；状态由主机固件管理的资源维护，OpenSFI 芯片代码没有其他资源来跨调用保存状态

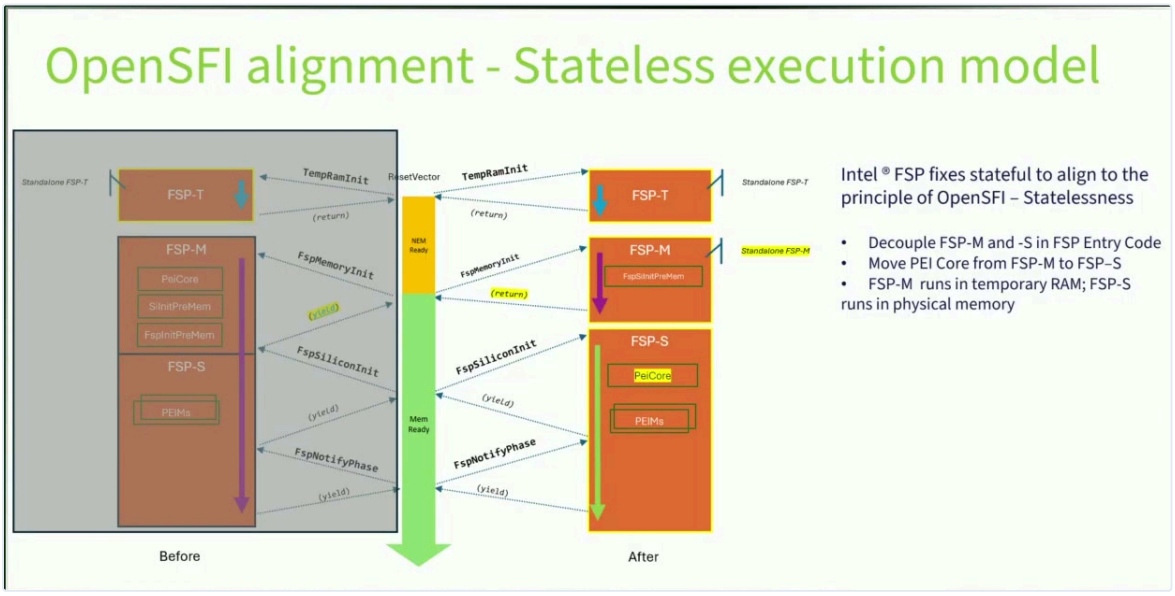
那么，我要讲的第一件事是：我们对 FSP 实现做了这些有意思的改动，它们带来不少好处。其中一些是——你知道，减少了临时 RAM 的使用之类，让事情更容易。但 openSFI 规范的原则之一，是关于无状态执行模型（stateless execution model）。而实际上，我们对 FSP 实现所做的这些改动，现在已经与那个无状态执行模型非常贴近了。因为以前我们有 PEI Core 和 FSP-M，然后在 FSP-M 和 FSP-S 里都有 PEIM，那些执行环境之间存在共享状态——就为了 PEI Core 的调度器、FV、通知之类的东西。通过把 FSP-M 改成完全不依赖 PEI、让它纯粹就是"把平台的内存打开"而没有 PEI Core 依赖，我们在这一点上现在非常干净。

所以"无状态"这个想法，意思是主机固件负责所有资源管理，包括所有内存和缓冲区的分配。当我们发起 openSFI 调用时，它负责分配传入的缓冲区和传出的缓冲区，并管理它们。我们对 FSP 做的这些改动，与 openSFI 规范的那些要求和原则非常契合。

> "So, the first thing I want to talk about is that, you know, we've done this interesting different changes to our implementation of FSP that have a number of advantages. Some of them are you know, some reduced use of temporary RAM and things like that that make it so it's easier. But the one of the things that's one of the principles from the Open SFI spec is about the stateless execution model. And really some of these changes we've made to our FSP implementation align very closely now with that stateless execution model because previously when we had the PEI core and FSPM and then we had PIMs in both FSPM and FSPS, there was shared state between those execution environments. Just for like the PEI core things like the dispatcher, the FVs, the notifications and and things like that. With changing FSPM to be not dependent upon PEI at all and it just being a pure let's go turn on memory in the platform without the PEI core dependency, we're now very very clean in that that respect. So this idea of being stateless, what that means is that the host firmware is responsible for all resource management and that includes all allocation of all memory and buffers. So when we make Open SFI calls, it's responsible for allocating the buffers for the buffers passed in and for the buffers passed out and and managing those. These changes we've made to the FSP very closely align with those requirements and that principle from the the Open SFI spec."

> **小结**　无状态 = 主机固件承担全部资源管理（含所有内存与缓冲区分配），芯片代码不在两次调用之间保存状态。FSP-M 剥离 PEI 依赖后，消除了 PEI Core 调度器/FV/通知带来的跨环境共享状态，因此天然贴合 openSFI 的这一原则。

08　对齐 openSFI：PEP 阶段与 VAL 层



OpenSFI alignment - Stateless execution model：左侧 Before 为 FSP-T/FSP-M/FSP-S 共享 PEI Core、经 FSPI 接口串行调用；中间新增 VAL 层（Vendor Abstraction Layer）与 Slim Boot 路径；右侧 After 将 FSP-M 的 PEI Core 移除、FSP-S 保留 PEI Core，FSP-M 在临时内存运行、FSP-S 在物理内存运行

如果我们看下一张图，这张图左边是以前的 FSP 设计：它经过一串调用，用 FSP-T 打开临时内存，再经过 FSP-M，走完那串调用；中间那个黄色和绿色代表主机固件在走那串调用的过程。那串调用就是在调用由

FSP-M 与 FSP-T 结合共享 PEI Core 所提供的服务。

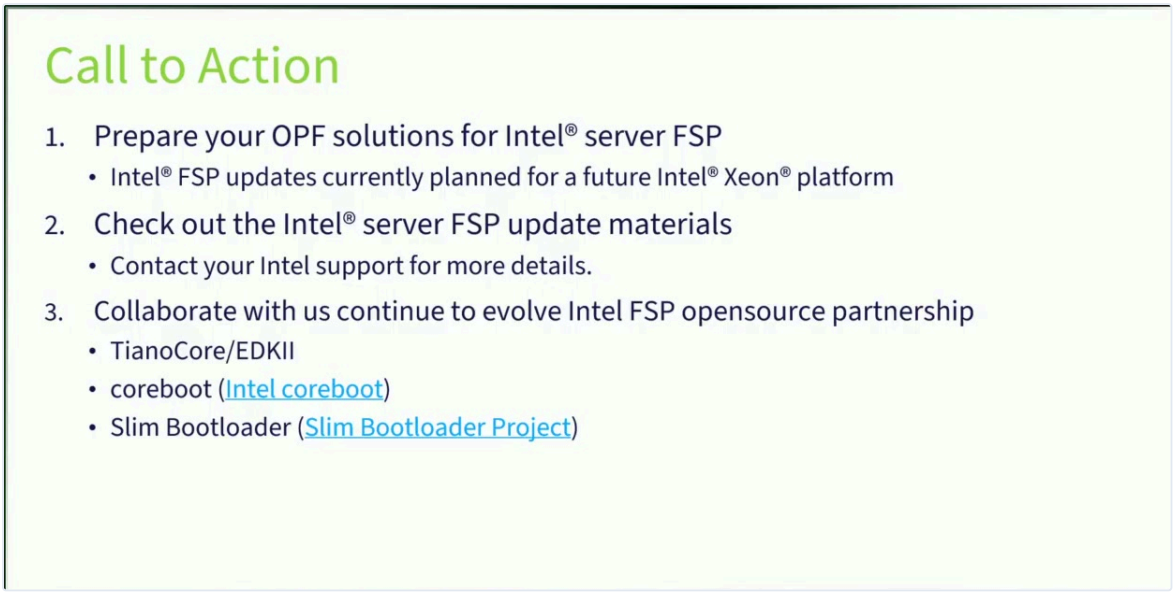
如果我们看右边的图，这又一次更贴近 openSFI 里发生的事：黄色区域调用 FSP-T 做临时内存、调用 FSP-M 打开内存，这与 openSFI 规范里的 PEP 阶段相对应。然后往下看，它开始调用 FSP 的芯片初始化（FSP initialize silicon，即 FSP-S），那正是通过 FSP 服务调用完成 SoC 初始化序列其余部分的地方。所以这说明了在 API 模式下是那样的。这也是我们转向 API 模式优先的原因之一——同样是为了对齐 openSFI。

openSFI 规范里提到的一个概念是 VAL 层（valve layer），即厂商抽象层（vendor abstraction layer）。所以你可以看到，在 FSP 和 FSP-M 都执行完、走完 PEP 阶段之后，我们接下来有一串通过 FSP-S 的调用。那些既有的 FSP API 和 openSFI 规范里定义的 API，这两套 API 之间关系相当接近。所以我们预期，在 FSP 之上实现出 openSFI 接口的 VAL 层，有望是一个相当薄的层——也许主要聚焦于为符合 openSFI 数据结构而必须做的一些数据操作，以及到 FSP 数据结构的翻译。这就是我们要去的方向；我们认为，随着接下来一两个季度 POC 工作的推进，我们将能够证明这是从 FSP 到 openSFI 的自然过渡。

> "And if we look at the the next picture here, this one shows on the left the previous FSP design where it goes through a sequence of of calls with FSPT to turn on temporary memory and FSPM and goes through that that sequence of calls with the the middle there with the yellow and the green is the represents the host firmware as it goes through that sequence of calls. And that sequence of calls is, you know, invoking the services that are produced by the combination of FSPM and T with the shared PEI core. If we look at the picture on the right, this once again more closely aligns with what's happening in Open SFI where the yellow area there where it invokes FSPT for temporary AM and FSPM to turn on memory. That aligns with the PEP stage inside of the open SFI specification. And then if you look down below where it starts invoking the initialize silicon calls for the FSP initialize silicon calls into FSPS, that's where it's completing the rest of the SOC initialization sequence through the FSP service calls. And so that's a you know, shows that that in API mode that's there. And this is one of the reasons we're moving to a you know, API mode first cuz once again to align with open SFI, one of the items that's mentioned in the open SFI spec is the concept of a valve layer, a vendor abstraction layer. So you can see that after we get through the PEP stage with FSP and FSPM both executed, we now have that sequence of calls through FSPS. Those existing FSP APIs and the APIs that are defined in the open SFI spec, there's a pretty close relationship between those two two sets of APIs. So we expect that the valve layer that that can be implemented to produce the open SFI interfaces on top of FSP will hopefully be a fairly thin layer, maybe focused on some of the data manipulation that has to occur to conform with the open SFI data structures and translations to the FSP ones. So that's where we're we're headed and we think that as far as we get going on the POC work here in the the next quarter or two, we'll be able to demonstrate that that is a a natural transition from FSP to open SFI."

> **小结**　映射关系清晰：FSP-T + FSP-M 对应 openSFI 的 PEP 阶段，FSP-S 的服务调用承接 SoC 初始化其余部分。由于两套 API 高度接近，VAL（厂商抽象层）可以做得很薄——主要做数据结构翻译。API 模式优先正是为了让这条映射成立；POC 计划在接下来一两个季度验证。

09　行动号召



Call to Action：①让 OPF 方案为 Intel 服务器 FSP 做好准备（更新计划面向未来 Xeon 平台）；②查阅 Intel 服务器 FSP 更新材料，详情联系 Intel 支持；③与 Intel 一起持续演进 FSP 开源合作（TianoCore/EDKII、coreboot、Slim Bootloader）

那么说到行动号召，这里的几项是：FSP 规范是 Intel 官网上的公开规范。你今天就可以围绕 FSP 准备你的实现，然后在未来完成到 openSFI 的过渡。Intel 与许多开源项目合作，比如 TianoCore（用于 UEFI 引导）、coreboot 和 Slim Bootloader，链接都在这里。如果你需要关于某个具体 Intel SoC 实现的 FSP 细节，可以联系你的 Intel 支持。呃，那么，就到这里。今天有问题吗？

> "So as far as call to action, you know, the the the items here are the FSP spec is a public spec on the Intel website. You can prepare your your implementations around FSP today and then and have this transition to open SFI in the in the future. Intel collaborates with many open source projects around things like TianoCore for the UEFI boot, core boot and slim boot. The the links are there. And if you need details about FSP for a specific you know Intel SOC implementation, you can contact your Intel support for that. Um so, with that, that's the the end. Are there there any questions for today?"

> **小结**　三步走：今天即可按公开的 FSP 规范做实现（未来再平滑过渡到 openSFI）；通过 Intel 支持获取具体 SoC 的 FSP 细节；在 TianoCore/EDKII、coreboot、Slim Bootloader 三条开源路径上继续共建。

附录　时间轴

时间	内容
00:00	开场，说明与 openSFI / UPCI 对齐
00:45	致谢页
01:03	议程

时间	内容
02:00	Intel FSP 概览：二进制封装、V2.5、API 与 dispatch 两种模式
03:30	四大支柱；强调规范不变、引导程序无需改写
05:12	高层架构对比：Xeon 6 与下一代 Xeon 的三项改动
09:18	无状态执行模型；主机固件承担资源管理
10:48	Before/After 流程图；PEP 阶段与 VAL 层
13:00	行动号召
13:50	结束